BÀI THỰC HÀNH MÔN HỌC MÁY

Dự báo doanh thu sản phẩm theo tuần

Cài đặt mô hình bằng NumPy và so sánh kết quả trên dữ liệu Olist.

Đơn vị: BRL | **Dữ liệu:** 1.198.182 mẫu sản phẩm–tuần | **Mô hình được chọn:** Two-Stage Hurdle

Lần chạy: 11/11/2026, chế độ full

1. Mục tiêu và cách đánh giá

Bài lab thực hành dự báo tổng doanh thu của từng sản phẩm trong tuần tiếp theo từ dữ liệu Olist. Mục tiêu học tập là tự cài đặt các mô hình bằng NumPy, xây dựng đặc trưng từ lịch sử và so sánh kết quả với các baseline đơn giản.

Mỗi mẫu là một cặp sản phẩm–tuần. Doanh thu được tính bằng tổng `price` của các món hàng, không cộng phí vận chuyển. Vì không có dữ liệu tồn kho, doanh thu quan sát được chưa thể xem là toàn bộ nhu cầu của khách hàng.

Trong 293.717 mẫu Test, có 286.058 mẫu không phát sinh doanh thu (97,39%) và 7.659 mẫu có doanh thu (2,61%). Do đó, cần đọc nhiều chỉ số cùng nhau: mô hình luôn dự báo 0 có thể đạt MAE thấp nhưng không dự báo được doanh thu ở các tuần có bán hàng.

MAE và RMSE (BRL)

$$\mathrm{MAE} = \frac{1}{N}\sum_i |s_i - \hat{s}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i (s_i - \hat{s}_i)^2}$$

MAE cho biết sai số tuyệt đối trung bình; RMSE nhạy hơn với sai số lớn. Bài lab dùng RMSE trên Validation để chọn mô hình. MSE cũng được lưu trong CSV, với đơn vị BRL bình phương.

Hệ số xác định R^2

$$R^2 = 1 - \frac{\sum_i (s_i - \hat{s}_i)^2}{\sum_i (s_i - \bar{s})^2}$$

Giá trị dương cho biết tổng sai số bình phương thấp hơn cách dự báo bằng trung bình của chính tập đánh giá. Điều này chưa nói lên mức cải thiện có ý nghĩa thống kê hay lợi ích kinh doanh.

WAPE

$$\mathrm{WAPE} = \frac{\sum_i |s_i - \hat{s}_i|}{\sum_i s_i}$$

Mẫu có doanh thu bằng 0 không gây phép chia riêng cho 0 như MAPE. Theo công thức, WAPE không xác định khi tổng doanh thu bằng 0. Code hiện dùng quy ước: nếu tổng thực tế không vượt 10^{-9}, trả 0 khi tổng sai số cũng không vượt ngưỡng đó, ngược lại trả 1. Đây là fallback của chương trình, không phải giá trị suy ra từ công thức. Các bảng ghi dạng tỷ số; ví dụ 1,9730 tương ứng khoảng 197,30%.

Điểm phạt dự báo thiếu và thừa

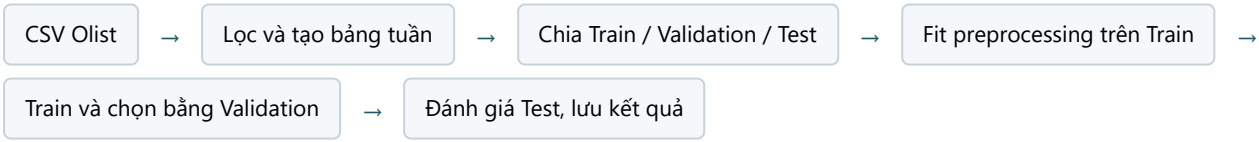
$$\mathrm{Cost_Per_Item} = \frac{1}{N}\sum_i \left[10\max(0, s_i - \hat{s}_i) + 2\max(0, \hat{s}_i - s_i)\right]$$

Hai trọng số 10 và 2 là giả định của bài lab. Chỉ số này được tính trên sai lệch doanh thu của mỗi mẫu sản phẩm–tuần, không phải chi phí kho thực tế hay chi phí cho mỗi món hàng.

2. Luồng xử lý của chương trình

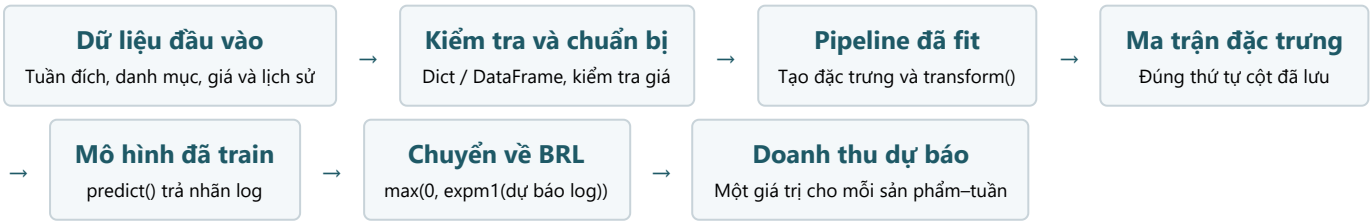
2.1. Luồng huấn luyện và đánh giá

`main.py` điều phối 10 bước: nạp dữ liệu, lọc giao dịch, tạo bảng tuần, vẽ EDA, chia dữ liệu và tiền xử lý, train, tuning, đánh giá Test, phân tích sai số, lưu mô hình và chạy ví dụ. Sơ đồ sau tóm tắt các nhóm xử lý chính.



2.2. Luồng suy luận (dự báo)

Luồng dưới đây áp dụng cho dự báo doanh thu tuần từ thông tin một sản phẩm hoặc nhiều sản phẩm trong CSV. Trước khi dự báo, ứng dụng tải model và pipeline từ `best_sales_model.pkl`.



1. **Chuẩn bị đầu vào:** người dùng cung cấp tuần dự báo và các giá trị lịch sử có trước tuần đó. Với pipeline tuần, nếu thiếu cột thời gian, hàm dùng tuần mặc định đã lưu; nếu thiếu một cột lịch sử, hàm thêm cột đó với giá trị 0. Đây là quy ước xử lý thiếu đầu vào, không có nghĩa lịch sử thực tế chắc chắn bằng 0.

2. **Biến đổi dữ liệu:** `pipeline.transform()` tạo đặc trưng, điền dữ liệu thiếu, mã hóa danh mục và chuẩn hóa bằng các tham số đã học trên Train. Trong snapshot này, đầu ra có 89 cột.
3. **Tính dự báo:** `model.predict()` nhận ma trận đặc trưng. Với Hurdle đang được lưu, mô hình nhân xác suất có doanh thu với dự báo doanh thu dương, sau đó trả kết quả dạng `log1p` để dùng chung giao diện.
4. **Trả kết quả:** `predict_sales()` dùng `expm1` và chặn dưới bằng 0, trả một mảng doanh thu BRL để ứng dụng hiển thị.

Nếu đầu vào đã là ma trận NumPy hoặc SciPy sparse đã tiền xử lý, hàm bỏ qua bước chuẩn bị và `transform()`, gọi thẳng model. Bên gọi cần bảo đảm ma trận có đúng đặc trưng và thứ tự cột như khi train.

Luồng suy luận chỉ sử dụng model và các tham số tiền xử lý đã lưu, không huấn luyện hay fit lại trên dữ liệu cần dự báo.

Luồng đánh giá tuần là *rolling-origin one-week-ahead*: ở mỗi tuần đích, có thể dùng lịch sử thực tế của các tuần trước đó, kể cả tuần trước nằm trong giai đoạn Validation hoặc Test. Đây không phải dự báo đồng thời nhiều tuần từ một mốc cố định.

Nhánh proxy bảy ngày từ giao dịch đầu dùng mô hình riêng. Phần marketing và giảm giá là ví dụ mô phỏng theo giả định, được trình bày ở mục 12.

3. Dữ liệu và cách chia theo thời gian

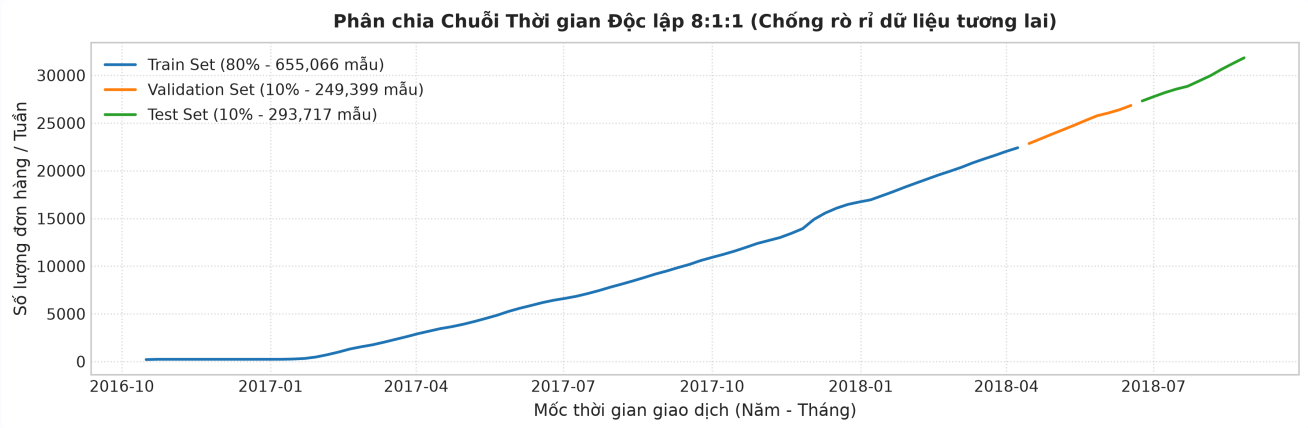
Dữ liệu nguồn gồm thông tin đơn hàng, các món trong đơn, sản phẩm, đánh giá, khách hàng và người bán. Pipeline ghép các bảng cần thiết và lọc theo trạng thái `delivered`. Đây là trạng thái ghi nhận cuối kỳ; bài lab chưa tái dựng trạng thái đơn hàng tại từng thời điểm dự báo.

Chia dữ liệu theo thứ tự các tuần với tỷ lệ mục tiêu 80/10/10. Trong lần chạy này, 98 tuần được chia thành 78 tuần Train, 10 tuần Validation và 10 tuần Test. Số sản phẩm thay đổi theo thời gian nên tỷ lệ số dòng khác tỷ lệ số tuần.

Bảng 3.1. Split trong pipeline_run_report.json

Tập	Số tuần	Ngày đầu tuần: từ → đến	Số dòng	Tỷ lệ dòng
Train	78	2016-10-10 → 2018-04-02	655.066	54,67%
Validation	10	2018-04-09 → 2018-06-11	249.399	20,81%
Test	10	2018-06-18 → 2018-08-20	293.717	24,51%

Train dùng để học tham số; Validation dùng để chọn cấu hình và mô hình; Test dùng để báo cáo kết quả sau lựa chọn. Việc giữ đúng thứ tự thời gian giúp hạn chế sử dụng thông tin tương lai, nhưng không loại bỏ được hạn chế của các trường dữ liệu hồi cứu.



Hình 3.1. Phân bố dữ liệu theo ba giai đoạn thời gian.

4. Bảng sản phẩm–tuần và biến mục tiêu

Code bỏ tuần đầu và tuần cuối của nguồn vì có thể chưa đủ ngày quan sát. Với mỗi sản phẩm còn lại, lưới tuần bắt đầu từ tuần bán đầu tiên đến tuần cuối của khoảng đã giữ. Tuần bán đầu được dùng làm lịch sử và bị loại khỏi tập mẫu dự báo; target đầu tiên nằm ở tuần kế tiếp. Tuần không có giao dịch được giữ lại với doanh thu bằng 0. Cách làm này giúp mô hình học cả những tuần có bán và không bán; sản phẩm chưa từng có giao dịch vẫn không xuất hiện trong bảng.

$$s_{p,t} = \sum_{j \in \text{các món của sản phẩm } p \text{ trong tuần } t} \text{price}_j$$

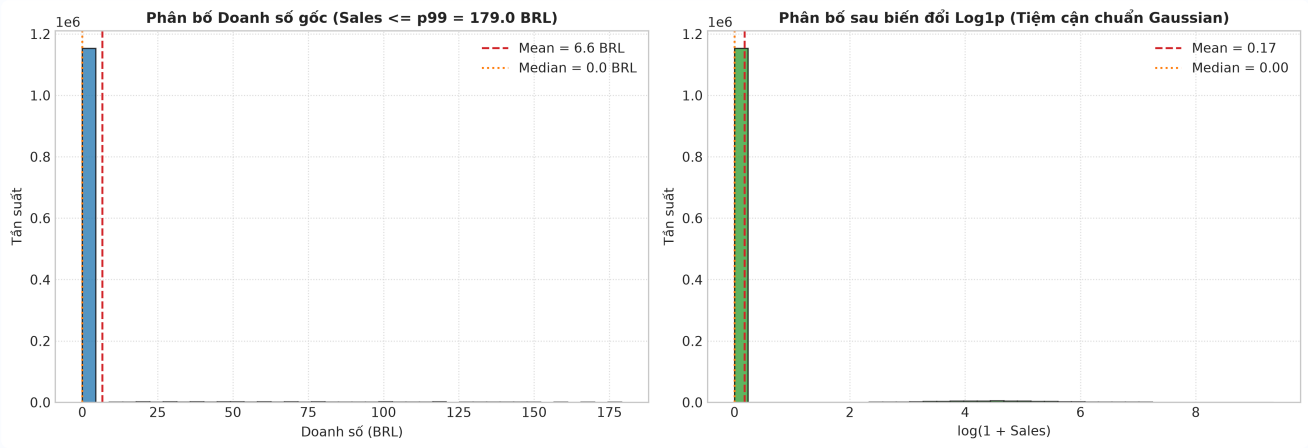
Tuần được tính từ thứ Hai đến Chủ nhật. Mỗi dòng món hàng đóng góp giá bán của nó; `order_item_id` là thứ tự món trong đơn, không phải số lượng.

Bảng thu được có 1.198.182 dòng, trong đó khoảng 96,21% có doanh thu bằng 0. Tỷ lệ 97,39% ở mục 1 chỉ tính riêng trên Test.

Biến đổi mục tiêu

$$y = \log(1 + s), \qquad \hat{s} = \max(0, \operatorname{expm1}(\hat{y}))$$

Logarithm nén khoảng giá trị của doanh thu lớn. Tuy vậy, tối ưu sai số trong không gian log không tương đương tối ưu sai số trên BRL. Vì vậy, bước chọn mô hình và báo cáo đều tính metrics sau khi chuyển về BRL.



Hình 4.1. Phân bố doanh thu tuần trong lần chạy được lưu.

5. Xây dựng đặc trưng

Trong lần chạy này, pipeline tạo 89 cột: 16 cột số và 73 cột danh mục sau One-Hot Encoding. Các biến lag và rolling dùng dữ liệu trước tuần đích. Số cột danh mục phụ thuộc vào dữ liệu Train.

Bảng 5.1: Bảng phân loại 89 đặc trưng trong ma trận thiết kế

Nhóm đặc trưng	Số lượng biến	Tên biến tiêu biểu	Cách tính và ý nghĩa
1. Trễ doanh số (Lags)	3 biến	`last_week_sales`, `sales_lag_2_weeks`, `sales_lag_4_weeks`	Doanh thu tại các tuần $t-1, t-2, t-4$.
2. Cửa sổ trượt (Rolling)	3 biến	`sales_mean_4_weeks`, `sales_std_4_weeks`, `last_week_units`	Trung bình động và độ biến động doanh số 4 tuần gần nhất; số lượng món đã bán tuần trước.
3. Giá bán	1 biến	`price`	Giá trung bình của tuần gần nhất có giao dịch trước tuần đích; điền tiếp giá lịch sử cho tuần không bán.
4. Đánh giá & Khách hàng	5 biến	`past_review_count`, `past_review_rating`, `customer_past_spend`, `customer_past_orders`, `customer_state_count`	Dùng đánh giá có trước tuần đích và lịch sử của nhóm khách mua gần nhất trong quá khứ. Không lấy đánh giá của tuần đích làm đặc trưng.
5. Thời gian	4 biến	`weeks_since_first_sale`, `month_sin`, `month_cos`, `is_peak_shopping_season`	Mã hóa chu kỳ 12 tháng: $\sin(2\pi m/12)$ và $\cos(2\pi m/12)$ để giữ tính liên tục giữa tháng 12 và tháng 1; cờ báo mùa cao điểm Black Friday & Giáng Sinh.
6. Danh mục sản phẩm	73 biến	`category_*` (One-Hot Encoded)	73 biến nhị phân đại diện cho 73 danh mục ngành hàng Olist (cama_mesa_banho, beleza_saude, informatica_acessorios...).

> **Lưu ma trận thưa:** Pipeline dùng `scipy.sparse.csr_matrix` cho đầu ra gồm phần số và One-Hot. Các mô hình có thể chuyển dữ liệu sang dense khi tính toán, nên dùng sparse không có nghĩa toàn bộ quá trình train đều tiết kiệm bộ nhớ. Lần chạy được lưu chưa cung cấp benchmark RAM hoặc mức tăng tốc để so sánh.

6. Tiền xử lý dữ liệu

`CustomDataPipeline` tạo đặc trưng rồi áp dụng các bước tiền xử lý. Trung vị, danh mục và tham số chuẩn hóa được học từ Train; Validation và Test dùng lại các giá trị này.

1. **Điền dữ liệu thiếu:** `CustomImputer` dùng trung vị cho cột số và mode cho cột phân loại. Nếu cột Train không có giá trị hợp lệ, fallback tương ứng là 0 và `other`. Các giá trị lịch sử mặc định còn được xử lý ở bước chuẩn bị đặc trưng.
2. **One-Hot Encoding:** danh mục mới chưa gặp khi fit được biểu diễn bằng vector 0.
3. **Chuẩn hóa:** tính $z = (x - \mu)/(\sigma + 10^{-8})$. Nếu độ lệch chuẩn trên Train bằng 0, code thay mẫu số chuẩn hóa bằng $1 + 10^{-8}$. Giá trị mới khác trung bình Train vì vậy không nhất thiết trở thành 0.
4. **Ghép cột:** kết hợp phần số và phần danh mục theo thứ tự `feature_names_` đã lưu.

Pipeline có cache để dùng lại ma trận và bộ tiền xử lý khi fingerprint còn hợp lệ. Cờ `--force-reprocess` yêu cầu xử lý lại dữ liệu.

7. Các mô hình và baseline

Bài lab tự cài đặt năm mô hình bằng NumPy, không dùng lớp mô hình của scikit-learn. Bốn baseline được dùng làm mốc so sánh trước khi đọc kết quả của các mô hình phức tạp hơn.

7.1. Baseline

- **Zero Baseline:** luôn dự báo 0 BRL.
- **Revenue Mean Baseline:** dự báo bằng doanh thu trung bình trên Train.
- **Mean Baseline:** dự báo trung bình nhãn log trên Train rồi chuyển về BRL.
- **Previous Week Baseline:** dùng doanh thu tuần liền trước.

7.2. Linear Regression

$$w = (X_b^T X_b + \alpha D)^+ X_b^T y, \qquad \alpha = 10^{-5}$$

Ở đây X_b gồm cột intercept; D là ma trận đường chéo với phần tử của intercept bằng 0, các phần tử còn lại bằng 1. Dấu $+$ là pseudo-inverse (`np.linalg.pinv`). Thành phần Ridge nhỏ giúp ổn định phép giải và không phạt intercept. Mô hình học trên nhãn log và có chặn dự báo theo Train. Sau phép mũ, sai số trên BRL vẫn có thể lớn.

Ở lần chạy này, Linear Regression có Test RMSE 5.511,6709 BRL, cao hơn các mô hình còn lại. Bảng metrics cho thấy kết quả kém trên BRL; để kết luận nguyên nhân ở từng mẫu cần xem riêng đầu vào và dự báo của mô hình.

7.3. Decision Tree

$$\Delta\mathrm{Var} = \mathrm{Var}(S) - \frac{|S_L|}{|S|}\mathrm{Var}(S_L) - \frac{|S_R|}{|S|}\mathrm{Var}(S_R)$$

Cây chọn điểm chia làm giảm phương sai nhiều nhất trong các ứng viên. Dự báo tại lá là trung bình nhãn log của các mẫu trong lá.

Khi một đặc trưng có hơn 16 giá trị phân biệt tại nút, code lấy 16 phân vị từ 5% đến 95% trên tập giá trị phân biệt làm ứng viên ngưỡng chia. Nếu có từ 2 đến 16 giá trị, code dùng trung điểm giữa các giá trị kề nhau; cột hằng được bỏ qua. Cách này giảm số ngưỡng cần thử nhưng có thể bỏ qua điểm chia tốt hơn. Bài lab chưa đo mức chênh lệch chất lượng hoặc thời gian so với duyệt mọi ngưỡng.

7.4. Random Forest

$$\hat{y}_{RF}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Mỗi cây lấy mẫu có hoàn lại với số lần lấy bằng số dòng Train. Một dòng có thể xuất hiện nhiều lần. Tại mỗi nút, cấu hình `sqrt` chọn khoảng $\sqrt{D}$ đặc trưng để xét chia.

Lấy trung bình nhiều cây nhằm giảm độ biến động so với một cây đơn. Trong thực nghiệm này, Random Forest không có RMSE thấp hơn Decision Tree.

7.5. Gradient Boosting

$$r_i^{(m)} = y_i - \hat{y}_i^{(m-1)}, \qquad \hat{y}^{(m)}(x) = \hat{y}^{(m-1)}(x) + \eta h_m(x)$$

Bắt đầu từ trung bình nhãn log, mỗi cây mới học phần dư của các cây trước. Số cây, độ sâu và learning rate ảnh hưởng cả chất lượng lẫn thời gian train.

7.6. Two-Stage Hurdle

Giai đoạn đầu dùng một cây hồi quy học nhãn 0/1 để ước lượng xác suất có doanh thu. Giai đoạn sau dùng cây hồi quy học nhãn log trên các mẫu có doanh thu dương.

$$\hat{s} = \hat{p}(s > 0 \mid x)\max(0, \operatorname{expm1}(\hat{y}_{pos})), \qquad \hat{y} = \log(1 + \hat{s})$$

Hàm `predict()` trả lại log để dùng chung giao diện với các mô hình khác. Khi đánh giá, kết quả được chuyển về BRL.

Lưu ý về phép nghịch đảo log:

$$\exp(\mathbb{E}[\log(1+S)]) - 1 \le \mathbb{E}[S]$$

Bất đẳng thức Jensen cho thấy nghịch đảo trung bình log không bằng trung bình doanh thu. Vì vậy, đầu ra kết hợp của Hurdle không mặc nhiên là kỳ vọng số học của doanh thu; cách biến đổi này cũng không bảo đảm loại bỏ sai số lớn.

8. Tuning và lựa chọn mô hình

Grid search thử 18 cấu hình của Random Forest, Decision Tree và Gradient Boosting. Mỗi cấu hình được fit trên Train và chấm bằng RMSE BRL trên 249.399 mẫu Validation.

Bảng 8.1. Cấu hình tốt nhất trong từng nhóm tuning

Nhóm	Tham số trong grid	Validation RMSE (BRL)
Random Forest	`{"n_estimators": 10, "max_depth": 8, "min_samples_split": 10}`	61,9221
Decision Tree	`{"max_depth": 8, "min_samples_leaf": 10}`	59,2288
Gradient Boosting	`{"n_estimators": 30, "max_depth": 3, "learning_rate": 0.1}`	61,2637

Decision Tree với `max_depth=8`, `min_samples_leaf=10` có RMSE thấp nhất trong grid và được ghi là `Tuned Model`. Sau đó, pipeline so sánh ứng viên này với chín mô hình/baseline đã train ban đầu.

Bảng 8.2. Lựa chọn mô hình từ validation_comparison.csv

Mô hình	Validation RMSE (BRL)	Được chọn
Zero Baseline	63,6067	
Revenue Mean Baseline	63,3996	
Mean Baseline	63,5862	
Previous Week Baseline	81,9053	
Linear Regression	11.211,0106	
Decision Tree	61,1664	
Random Forest	63,1106	
Gradient Boosting	62,8054	
Two-Stage Hurdle	55,9393	Có
Tuned Model	59,2288	

Two-Stage Hurdle được chọn với Validation RMSE **55,9393 BRL**. Mô hình này có độ sâu hai cây lần lượt là 5 và 6, `min_samples_leaf=10`. Hurdle không nằm trong 18 cấu hình grid search nêu trên. Test không được dùng để chọn mô hình.

9. Cấu hình và thời gian huấn luyện

Snapshot kết thúc ngày 11/11/2026 lúc 22:49:49, dùng chế độ `full` với 655.066 dòng Train, seed 42. Tổng thời gian ghi nhận là 2.180,89 giây; riêng bước tuning là 1.800,62 giây. Các thời gian dưới đây thuộc lần chạy này và có thể thay đổi theo máy.

Bảng 9.1. Cấu hình và thời gian fit trước tuning

Mô hình	Tham số chính	Fit time (giây)
Linear Regression	`solver=normal_equation, learning_rate=0.01`	0,61
Decision Tree	`max_depth=5, min_samples_leaf=10`	17,51
Random Forest	`n_estimators=15, max_depth=6, min_samples_leaf=10`	67,78
Gradient Boosting	`n_estimators=15, learning_rate=0.1, max_depth=3, min_samples_leaf=5`	235,84
Two-Stage Hurdle	`classifier_max_depth=5, regressor_max_depth=6, min_samples_leaf=10`	21,35

Random Forest ban đầu dùng 15 cây, khác với cấu hình tốt nhất trong grid. Chưa có số đo RAM trong report của lần chạy này.

10. Kết quả trên tập Test

Bảng dưới lấy từ `results/metrics/model_comparison.csv`, gồm chín mô hình/baseline ban đầu và một ứng viên sau tuning. Tất cả được đánh giá trên cùng 293.717 mẫu Test. `Tuned Model` ở đây là Decision Tree đã nêu ở mục 8.

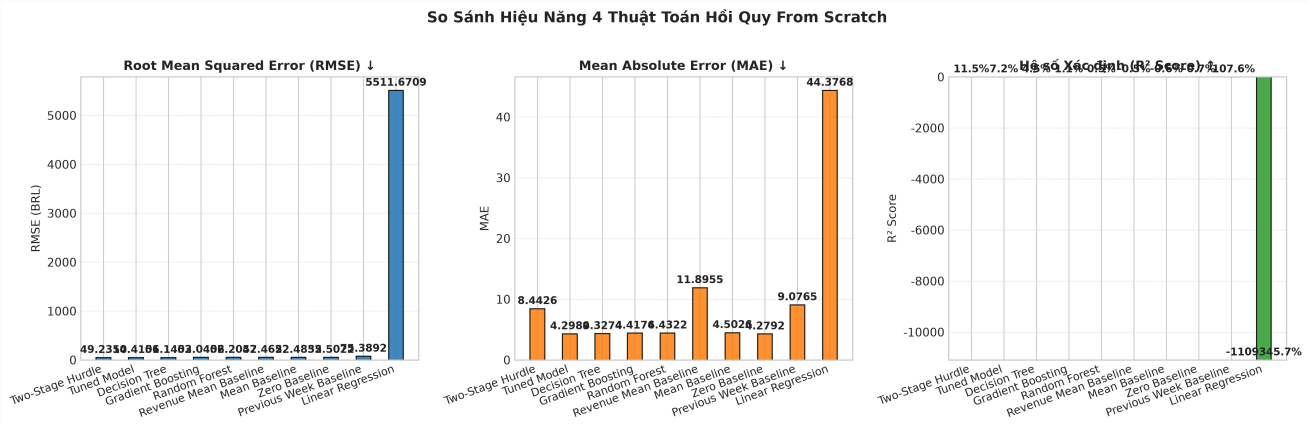
Bảng 10.1. Kết quả Test; tên mô hình giữ theo CSV

Mô hình	RMSE (BRL)	MAE (BRL)	R^2	WAPE	Điểm phạt/mẫu	Fit time (s)
Two-Stage Hurdle	49,2314	8,4426	0,1148	1,9730	44,7952	21,35
Tuned Model	50,4106	4,2989	0,0719	1,0046	40,6158	0,00
Decision Tree	51,1403	4,3274	0,0449	1,0113	41,2640	17,51
Gradient Boosting	52,0406	4,4176	0,0109	1,0323	42,4902	235,84
Random Forest	52,2047	4,4322	0,0047	1,0357	42,6916	67,78
Revenue Mean Baseline	52,4622	11,8955	-0,0052	2,7798	56,3489	0,00
Mean Baseline	52,4835	4,5026	-0,0060	1,0522	43,1895	0,00
Zero Baseline	52,5022	4,2792	-0,0067	1,0000	42,7919	0,00
Previous Week Baseline	75,3892	9,0765	-1,0757	2,1211	42,4533	0,00
Linear Regression	5.511,6709	44,3768	-11.093,4570	10,3704	120,7011	0,61

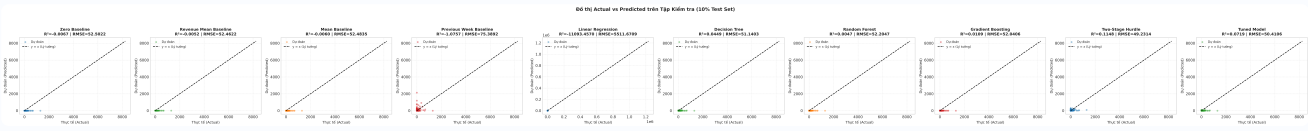
Fit time bằng 0 của Tuned Model là giá trị được code điền khi lập bảng; không có nghĩa tuning không tốn thời gian. Thời gian tổng của bước tuning nằm ở mục 9. Các baseline cũng được ghi thời gian bằng 0 trong bảng.

Hurdle có RMSE thấp nhất trong bảng (49,2314 BRL), nhưng MAE 8,4426 BRL cao hơn Zero Baseline (4,2792 BRL). Hai chỉ số phản ánh hai cách tính sai số khác nhau; kết quả này chưa chứng minh Hurdle phát hiện tốt các tuần có doanh thu đột biến. Bài lab chưa báo cáo precision/recall cho nhiệm vụ đó.

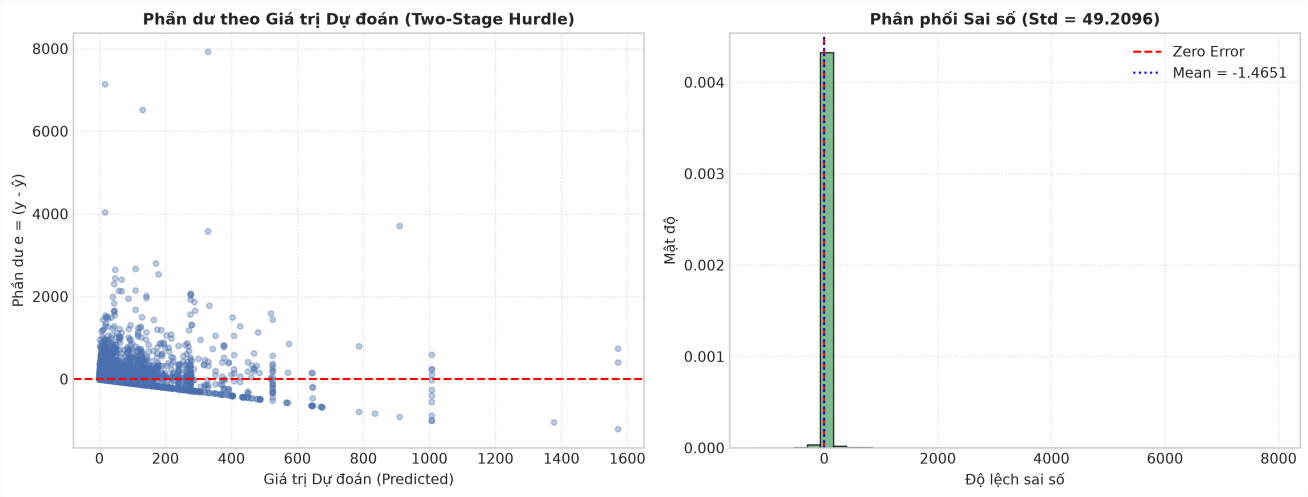
Tuned Model có điểm phạt thấp nhất, 40,6158 trên mỗi mẫu, so với 44,7952 của Hurdle. Đây là so sánh theo trọng số giả định 10/2, chưa phải bằng chứng tiết kiệm chi phí kho. Mô hình được lưu vẫn là Hurdle theo tiêu chí Validation RMSE đã dùng.



Hình 10.1. So sánh các chỉ số đánh giá trên Test.



Hình 10.2. Doanh thu thực tế và dự báo; mỗi mô hình hiển thị 2.000 mẫu lấy ngẫu nhiên với seed 42. Nhãn "10% Test Set" có sẵn trong ảnh chỉ tỷ lệ tuần mục tiêu; Test thực tế gồm 10/98 tuần và 24,51% số dòng.



Hình 10.3. Phân bố và độ lớn phần dư; dùng để quan sát các sai số còn lại.

11. Phân tích các mẫu dự báo sai

Năm dòng dưới có sai số tuyệt đối lớn nhất trong `worst_predictions.csv`, được tính cho mô hình đã chọn. Sample Index là vị trí mẫu trong tập đánh giá, không phải mã đơn hàng.

Bảng 11.1. Năm mẫu sai số lớn nhất

Hạng	Sample Index	Thực tế (BRL)	Dự báo (BRL)	Sai số tuyệt đối (BRL)	Sai số tương đối
1	227704	8.251,00	328,47	7.922,53	96.0%
2	86202	7.160,00	16,64	7.143,36	99.8%
3	106746	6.645,70	130,14	6.515,56	98.0%
4	166054	4.059,00	16,64	4.042,36	99.6%
5	222379	4.620,00	909,84	3.710,16	80.3%

Cả năm mẫu đều bị dự báo thấp hơn thực tế. Ví dụ, mẫu 227704 có doanh thu 8.251,00 BRL nhưng dự báo chỉ 328,47 BRL. Các mẫu này cho thấy mô hình còn bỏ sót một phần doanh thu lớn, dù RMSE tổng thể thấp hơn baseline.

Bảng sai số không đủ để xác định nguyên nhân là mua sỉ, khuyến mãi hay hết hàng. Dữ liệu đang dùng không có nhật ký chiến dịch và tồn kho để kiểm chứng các giả thuyết này. Bước phân tích tiếp theo nên ghép mẫu sai số với

sản phẩm, tuần và chuỗi doanh thu trước đó.

12. Lưu mô hình và ứng dụng minh họa

`best_sales_model.pkl` lưu một dictionary gồm model tuần, pipeline đã fit, model cold-start nếu có, cùng metadata. `SalesPredictor` là lớp dùng các thành phần này; bản thân file pickle không phải một đối tượng SalesPredictor.

`app.py` dùng Streamlit để minh họa năm chức năng theo thứ tự: một sản phẩm; nhiều sản phẩm từ CSV; mô phỏng giảm giá; bảy ngày sau lần bán đầu; mô phỏng marketing.

12.1. Proxy bảy ngày từ lần bán đầu

Nhánh này dự báo tổng doanh thu trong bảy ngày tính từ giao dịch đầu tiên, **bao gồm giao dịch đầu đã biết**. Giá đầu vào là giá giao dịch đó. Đây chưa phải đánh giá dự báo trước ngày niêm yết và không bao gồm sản phẩm chưa bán được hàng.

Manifest ghi 23.428 sản phẩm Train và 8.110 sản phẩm Test, với mốc chia 24/04/2018. Có 483 mẫu Train bị loại vì cửa sổ nhãn vượt mốc chia. Chỉ giữ sản phẩm có đủ bảy ngày quan sát.

Bảng 12.1. Kết quả proxy từ cold_start_evaluation.csv

Mô hình	MAE (BRL)	RMSE (BRL)	R^2	Số mẫu Test
CategoryMedianColdStartBaseline	126,6543	313,9562	0,0022	8.110
SingleUnitPriceBaseline	23,0145	120,9134	0,8520	8.110
ColdStartProductRegressor	39,7939	135,4682	0,8142	8.110

Baseline dự báo bằng giá một đơn vị có RMSE 120,9134 BRL, thấp hơn cây cold-start (135,4682 BRL). Trong Test, 85,47% sản phẩm chỉ bán một đơn vị trong cửa sổ bảy ngày; với những mẫu đó, target bằng giá giao dịch đầu đã biết. Vì vậy, không thể dùng kết quả này để kết luận cây dự báo tốt cho sản phẩm chưa từng bán.

Cây cold-start được lưu là ứng viên có cấu hình định trước, không được chọn bằng Test. Metadata sản phẩm và trạng thái delivered là snapshot hồi cứu; khả năng sẵn có tại thời điểm dự báo vẫn là giả định.

12.2. Mô phỏng marketing và giảm giá

Dữ liệu train không có lịch sử chiến dịch quảng cáo để học tác động marketing. Nhánh cơ sở không ước lượng doanh thu tăng thêm hoặc ROI. Nhánh uplift dùng hàm phản hồi logarit với hệ số giả định, chỉ để quan sát kết quả khi thay đổi tham số. Các chỉ số trả về từ mô phỏng không phải hiệu quả chiến dịch đo được.

Ví dụ giảm giá tính doanh thu bằng giá sau giảm nhân số lượng giả định và tính số lượng cần bán để giữ doanh thu tham chiếu. Số lượng là đầu vào của kịch bản, chưa được mô hình ước lượng từ phản ứng của khách hàng với giá.

13. Kiểm thử và cách chạy lại

Thư mục `tests/` có các kiểm thử về mô hình, tiền xử lý, chia thời gian, cold-start, lưu/tải và xuất report. Lần cập nhật tài liệu này đối chiếu mã nguồn cùng artifacts đã lưu; không chạy lại toàn bộ bộ test hay huấn luyện. Vì vậy, báo cáo không gán kết quả “passed” hoặc thời gian chạy cho bộ test hiện tại.

Các lệnh chạy từ thư mục gốc dự án

```
# Chạy các kiểm thử
.venv/Scripts/python.exe -m pytest tests/

# Chạy pipeline, dùng cache nếu còn hợp lệ
.venv/Scripts/python.exe main.py

# Xử lý lại từ dữ liệu nguồn rồi chạy pipeline
.venv/Scripts/python.exe main.py --force-reprocess

# Chạy nhanh để kiểm tra luồng; không dùng để tái lập số liệu full ở trên
.venv/Scripts/python.exe main.py --fast-mode --skip-eda --skip-tuning

# Mở ứng dụng minh họa
.venv/Scripts/python.exe -m streamlit run app.py
```

Cần có môi trường và dữ liệu nguồn của dự án trước khi chạy. Pipeline cập nhật CSV, JSON, biểu đồ, model và phần kết quả trong README. Report HTML này là tài liệu tĩnh, cần được đối chiếu lại sau lần chạy mới.

14. Hạn chế và hướng cải thiện

Kết quả hiện tại giúp so sánh các cách làm trong phạm vi bài lab. Những điểm còn hạn chế gồm:

- **Dữ liệu hồi cứu:** trạng thái delivered và metadata sản phẩm chưa có lịch sử đầy đủ để xác minh tại từng thời điểm dự báo. Ảnh hưởng của hạn chế này lên metrics chưa được đo.
- **Tuần không bán:** chưa phân biệt được không có nhu cầu, hết hàng hay sản phẩm ngừng bán. Bảng tuần chỉ bắt đầu khi sản phẩm đã có giao dịch.
- **Sai số doanh thu lớn:** các mẫu ở mục 11 vẫn bị dự báo thấp. Có thể phân tích thêm theo nhóm doanh thu và lịch sử sản phẩm để tìm hướng cải thiện.
- **Phạm vi đánh giá:** kết quả dùng một cách chia thời gian; chưa có khoảng tin cậy hoặc đánh giá trên nhiều giai đoạn. RMSE thấp nhất trong bảng không có nghĩa mô hình phù hợp cho mọi mục tiêu.
- **Cold-start và marketing:** proxy bắt đầu từ giao dịch đã biết; marketing và giảm giá dùng giả định. Cần dữ liệu phù hợp trước khi đánh giá dự báo trước bán hoặc tác động của chiến dịch.
- **Ứng dụng minh họa:** Streamlit phục vụ chạy thử cục bộ. Nếu mở rộng bài lab, nên ưu tiên kiểm tra dữ liệu đầu vào và chất lượng dự báo trên các giai đoạn mới trước.

15. Các tệp dùng để đối chiếu

Các tệp dưới đây là nguồn đối chiếu cho nội dung và số liệu trong báo cáo.

Bảng 15.1. Mã nguồn và kết quả đã lưu

Đường dẫn	Nội dung
`main.py`	Điều phối pipeline; hỗ trợ bốn cờ CLI nêu ở mục 13.

Đường dẫn	Nội dung
`src/config.py`	Đường dẫn, đặc trưng và cấu hình.
`src/data/builder.py`	Tạo bảng tuần và dữ liệu proxy.
`src/data/preprocessing.py`	Pipeline tiền xử lý.
`src/models/linear.py`	Linear Regression.
`src/models/tree.py`	Decision Tree.
`src/models/ensemble.py`	Random Forest và Gradient Boosting.
`src/models/hurdle.py`	Two-Stage Hurdle.
`src/evaluation/metrics.py`	Công thức các chỉ số đánh giá.
`results/metrics/pipeline_run_report.json`	Snapshot trước, trong và sau train; thời gian và cấu hình.
`results/metrics/validation_comparison.csv`	Điểm Validation và mô hình được chọn.
`results/metrics/model_comparison.csv`	Kết quả Test.
`results/metrics/tuning_history.csv`	18 cấu hình đã thử.
`results/metrics/worst_predictions.csv`	20 mẫu có sai số lớn; không xác nhận nguyên nhân sai số.
`results/metrics/cold_start_manifest.json`	Mốc chia và giới hạn của proxy.
`results/metrics/cold_start_evaluation.csv`	So sánh cây cold-start với baseline.
`tests/`	Các kiểm thử của dự án.
`app.py`	Ứng dụng Streamlit minh họa.

Báo cáo bài lab Học máy — cập nhật ngày 11/11/2026 theo snapshot kết thúc lúc 22:49:49. Số liệu được làm tròn khi trình bày; các giới hạn đánh giá được ghi ở mục 14.